Materials for List 1: PyTorch & Training Fundamentals

Large Language Models · Wrocław University of Science and Technology · Winter Term 2026/2027

Background for the tasks of List 1. Section numbers match the task numbers of the list. You can read it as a PDF, or run the notebook and experiment with the examples. The examples deliberately use different data from the tasks.

The material follows Appendix A of Raschka's *Build a Large Language Model (From Scratch)* (the same content as his free tutorial *PyTorch in One Hour*); Sections 2.3 and 2.4 follow the chapter "N-gram Language Models" of Jurafsky & Martin, *Speech and Language Processing*.

```
import math

import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn.functional as F
from torch import nn
from torch.utils.data import DataLoader, Dataset, TensorDataset

print("PyTorch", torch.__version__)
```

```
PyTorch 2.13.0+cpu
```

0.1 Reading

- **Before Lab 1:** BLLM Appendix A, Sections A.1–A.5, or Raschka, *PyTorch in One Hour*, Sections 1–5
- **Before Lab 2:** Sections A.6–A.8 (Sections 6–8 of the tutorial); Jurafsky & Martin, *Speech and Language Processing*, chapter "N-gram Language Models": bigrams, smoothing, perplexity
- **Optional:** Karpathy, "The spelled-out intro to language modeling: building makemore" (video; Tasks 2.3–2.4 follow the same idea)

0.2 Setup

Python 3.10 or newer with `torch` and `matplotlib` (`pip install torch jupyterlab matplotlib`). A GPU is not needed; Google Colab also works; everything in this list runs in about a minute on a laptop CPU.

Notebook or scripts. In a notebook, cells share one Python process: a variable defined in one cell is visible in all later ones, and running cells out of order is the most common source of confusing errors. Before you submit, restart the kernel and run everything from the top. In scripts, put the four reusable functions in one module (say `list01.py`) and import them from the other scripts.

What PyTorch is. Three things in one library:

1. a **tensor library**: NumPy-like arrays that can also live on a GPU;
2. **autograd**: it records the operations you apply to tensors and computes gradients of any scalar result automatically;
3. **deep-learning building blocks**: layers and models (`torch.nn`), optimisers (`torch.optim`) and data loading (`torch.utils.data`).

Sections 1.1, 1.2 and 1.4–2.2 introduce these three parts in this order.

Lab 1: Tensors, Autograd & the Training Loop

1.1 Tensors

What a tensor is

A **tensor** is a multi-dimensional array of numbers of one type. A tensor of **rank** r (also *order*, or number of *dimensions*) is an element of $\mathbb{R}^{n_1 \times \cdots \times n_r}$:

rank	name	example shape	typical use
0	scalar	`()`	a loss value
1	vector	`(n,)`	a bias, one feature vector
2	matrix	`(m, n)`	a weight matrix, a batch of feature vectors
3	3D tensor	`(B, T, d)`	a batch of B sequences of T tokens, each a d-dimensional vector

The rank counts **axes**, not entries: a vector with 1000 entries is still rank 1. Three attributes describe every tensor: `.shape` (the sizes $n_1, \ldots, n_r$), `.dtype` (the type of the entries) and `.device` (where it is stored: CPU or GPU).

```
s = torch.tensor(3.0)                       # rank 0
v = torch.tensor([1.0, 2.0, 3.0])           # rank 1
M = torch.tensor([[1, 2], [3, 4]])          # rank 2
T = torch.zeros(2, 3, 4)                    # rank 3, filled with zeros
for t in [s, v, M, T]:
    print(t.ndim, tuple(t.shape), t.dtype, t.device)

# other constructors
print(torch.arange(5), torch.ones(2, 2), torch.randn(2, 3), sep="\n")
```

```
0 () torch.float32 cpu
1 (3,) torch.float32 cpu
2 (2, 2) torch.int64 cpu
3 (2, 3, 4) torch.float32 cpu
tensor([0, 1, 2, 3, 4])
tensor([[1., 1.],
        [1., 1.]])
tensor([[ 0.7781,  0.9331,  0.3731],
        [-1.3017,  0.4612,  0.0839]])
```

Indexing and slicing work as in NumPy: `M[0]` is the first row, `M[:, 1]` the second column, `M[M > 2]` the entries larger than 2. `torch.randn` samples from $\mathcal{N}(0,1)$, `torch.rand` from the uniform distribution on $[0,1)$.

Data types and floating point

dtype	bits	used for
`torch.float32` (`float`)	32	default for real numbers; parameters, activations
`torch.float64` (`double`)	64	numerical checks; NumPy's default
`torch.float16` , `torch.bfloat16`	16	fast training on GPUs (later in the course)
`torch.int64` (`long`)	64	indices, class labels, token IDs
`torch.bool`	8	masks

PyTorch infers `int64` from Python integers and `float32` from Python floats. Convert with `.to(torch.float64)` , `.double()` , `.float()` or `.long()` . When an operation mixes types, the result is **promoted** to a common type (`torch.result_type(a, b)` tells you which).

A floating-point number is stored as $\pm m \cdot 2^{e}$ with a significand m of fixed length: 24 bits in `float32` , 53 bits in `float64` . The gap between 1 and the next representable number, the **machine epsilon**, is

$$\varepsilon_{\text{float32}} = 2^{-23} \approx 1.19 \times 10^{-7}, \qquad \varepsilon_{\text{float64}} = 2^{-52} \approx 2.22 \times 10^{-16}.$$

Every arithmetic operation rounds its result to the nearest representable number, which means a *relative* error of up to $\varepsilon/2$. So `float32` keeps about 7 significant decimal digits and `float64` about 16. Adding a number that is smaller than about ε times another one does nothing at all:

```
for dt in [torch.float32, torch.float64]:
    one = torch.tensor(1.0, dtype=dt)
    print(dt, "eps =", torch.finfo(dt).eps, "| 1 + 1e-10 == 1:", (one + 1e-10 == one).ite
```

```
torch.float32 eps = 1.1920928955078125e-07 | 1 + 1e-10 == 1: True
torch.float64 eps = 2.220446049250313e-16 | 1 + 1e-10 == 1: False
```

Deep learning uses `float32` because it halves the memory and is much faster on GPUs, and because training is noisy anyway: errors of 10^{-7} do not matter. Numerical *checks* (like the gradient check of Task 1.2) are done in `float64` .

Memory layout: strides and contiguity

A tensor's entries are stored in **one flat block of memory**, whatever its shape. The shape says how to read the block; the **strides** say how far to jump in the block to move one step along each axis. For a tensor with strides $(s_0, s_1, \ldots)$, entry $(i_0, i_1, \ldots)$ is stored at position

$$\text{offset} = i_0 s_0 + i_1 s_1 + \ldots$$

A 3×4 matrix created normally is stored row by row (**row-major** order):

```
A = [[ 0,  1,  2,  3],
     [ 4,  5,  6,  7],          memory:  0 1 2 3 4 5 6 7 8 9 10 11
     [ 8,  9, 10, 11]]
```

Moving one column to the right moves one place in memory; moving one row down moves four places. So the strides are $(4, 1)$, and entry $(2, 1)$ is at offset $2 \cdot 4 + 1 \cdot 1 = 9$, which holds the value 9.

```
A = torch.arange(12).reshape(3, 4)
print(A)
print("strides:", A.stride())
print("A[2, 1] =", A[2, 1].item(), "at offset", 2 * A.stride(0) + 1 * A.stride(1))
```

```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
strides: (4, 1)
A[2, 1] = 9 at offset 9
```

Many operations change only the shape and strides and leave the memory alone. Such a result is a **view**: it shares memory with the original, so it costs nothing to create and a change to one is visible in the other. Transposing is the standard example. `A.T` has shape $(4, 3)$ and strides $(1, 4)$: moving along a row of `A.T` (down a column of `A`) now jumps four places in memory. No data was copied.

```
B = A.T
print("A.T strides:", B.stride(), "| same memory:", B.data_ptr() == A.data_ptr())
B[0, 0] = 100                       # changes A as well
print(A[0, 0].item())
A[0, 0] = 0
```

```
A.T strides: (1, 4) | same memory: True
100
```

A tensor is **contiguous** if its entries lie in memory in the order you read them (row by row), with no gaps. `A` is contiguous; `A.T` is not, because reading `A.T` row by row means reading memory positions 0, 4, 8, 1, 5, 9, ... `.is_contiguous()` tells you which case you have.

This is what separates the two ways of changing a shape:

- `view(new_shape)` **never copies**. It only computes new strides for the same memory, which is possible only when the requested shape can be read from the memory with constant jumps. On a contiguous tensor this always works; on a non-contiguous one it often does not, and `view` raises an error.
- `reshape(new_shape)` returns a view when one exists and **copies** the data into a new contiguous block otherwise.
- `.contiguous()` makes that copy explicitly (and does nothing if the tensor is already contiguous).

Both need the same number of entries before and after; one size may be `-1`, meaning "whatever is left". Some operations (`.T`, `.transpose`, `.permute`, slicing with steps) produce non-contiguous views; arithmetic produces new contiguous tensors.

```
print(A.view(2, 6).shape, A.view(-1).shape)          # fine: A is contiguous
C = A[:, ::2]                                        # every second column: a non-contigu
print("C strides:", C.stride(), "contiguous:", C.is_contiguous())
print(C.reshape(-1))                                 # reshape copies when it has to
print(C.contiguous().view(-1))                       # or copy explicitly, then view
```

```
torch.Size([2, 6]) torch.Size([12])
C strides: (4, 2) contiguous: False
tensor([ 0,  2,  4,  6,  8, 10])
tensor([ 0,  2,  4,  6,  8, 10])
```

Why care? Later in the course, attention reshapes tensors of shape (B, T, d) into $(B, h, T, d/h)$ and back, with transposes in between. Knowing which operations copy explains both the errors you will see and the speed of the code.

Operations

- **Element-wise**: `+`, `-`, `*`, `/`, `**`, `torch.exp`, `torch.log`, ... act entry by entry and need matching (or broadcastable, see below) shapes. `A * B` is the element-wise (Hadamard) product, *not* the matrix product.
- **Matrix product**: `A @ B` (or `torch.matmul`) for $A \in \mathbb{R}^{m \times k}$, $B \in \mathbb{R}^{k \times n}$ gives $m \times n$. For tensors with more than two dimensions the last two are multiplied and the leading ones are treated as a batch: $(B, m, k) \mathbin{@} (B, k, n) \to (B, m, n)$. A vector on the right is treated as a column: $(m, k) \mathbin{@} (k,) \to (m,)$.
- **Reductions**: `sum`, `mean`, `max`, `argmax`, ... over a dimension: `M.sum(dim=0)` sums over rows (one value per column). `keepdim=True` keeps the reduced axis with size 1, which is handy for broadcasting the result back.

Broadcasting

Element-wise operations on tensors of different shapes follow the **broadcasting** rule (the same as NumPy's):

1. Align the two shapes at the **right** end. If one shape is shorter, pad it on the left with 1s.
2. Compare the sizes axis by axis. They are compatible if they are **equal** or **one of them is 1**.
3. A size-1 axis is stretched (without copying) to the other size. The result has the larger size on every axis.

Example: $(4, 1, 3) + (5, 1)$.

	axis 0	axis 1	axis 2
first	4	1	3
second (padded)	1	5	1
result	4	5	3

```
x = torch.zeros(4, 1, 3)
y = torch.zeros(5, 1)
print((x + y).shape)

M = torch.arange(6.0).reshape(2, 3)
```

```
print(M - M.mean(dim=0))                    # (2, 3) - (3,): subtract the column means
print(M / M.sum(dim=1, keepdim=True))       # (2, 3) / (2, 1): normalise each row
```

```
torch.Size([4, 5, 3])
tensor([[-1.5000, -1.5000, -1.5000],
        [ 1.5000,  1.5000,  1.5000]])
tensor([[0.0000, 0.3333, 0.6667],
        [0.2500, 0.3333, 0.4167]])
```

A classic bug: a vector of shape $(N,)$ combined with a column of shape $(N, 1)$ broadcasts to an (N, N) matrix instead of failing. For example, if predictions have shape $(N, 1)$ and targets shape $(N,)$, `((pred - target) ** 2).mean()` silently averages N^2 wrong numbers. Print shapes whenever a result looks odd.

```
pred = torch.zeros(4, 1)
target = torch.ones(4)
print((pred - target).shape)   # (4, 4), not (4,)
```

```
torch.Size([4, 4])
```

NumPy and PyTorch

	NumPy	PyTorch
default float type	`float64`	`float32`
array → tensor, sharing memory		`torch.from_numpy(a)`
array → tensor, copying		`torch.tensor(a)`
tensor → array, sharing memory		`t.numpy()` (CPU tensors without gradient tracking)
GPU, autograd	no	yes

A tensor made from a NumPy array keeps the array's dtype, so `torch.from_numpy(np.zeros(3))` is `float64`. Mixing it with `float32` parameters later either fails or silently promotes; convert explicitly with `.float()` when you move data into a model. `t.numpy()` fails for a tensor that requires gradients: use `t.detach().numpy()`.

Devices

`t.to("cuda")` copies a tensor to an NVIDIA GPU (`"mps"` on Apple Silicon). All inputs of an operation must be on the same device. Nothing in List 1 needs a GPU.

PyTorch you need. `torch.tensor`, `zeros`, `ones`, `arange`, `randn`; `.shape`, `.ndim`, `.dtype`, `.stride()`, `.is_contiguous()`, `.data_ptr()`; `.view`, `.reshape`, `.contiguous`, `.T`; `@`, `*`; `torch.from_numpy`, `.numpy()`; `torch.finfo(dtype).eps`.

1.2 Autograd and a gradient check

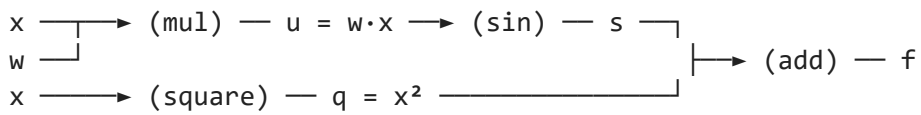
Why gradients

Training a model means minimising a loss $L(\theta)$ over its parameters θ, which can be millions of numbers. Gradient descent needs $\nabla_\theta L$, the derivative of one scalar with respect to every parameter.

Deriving it by hand for every model is impractical, so PyTorch computes it automatically. This is **automatic differentiation** (autograd).

Computation graphs

Any computation can be written as a **computation graph**: a directed acyclic graph with one node per elementary operation, edges carrying intermediate results from the inputs to the output. For $f(x, w) = \sin(wx) + x^2$:



The **inputs and parameters** (x, w) are the *leaves*; f is the *output*.

PyTorch builds this graph on the fly while you compute. It does so for every tensor that depends on a leaf created with `requires_grad=True`. Every non-leaf result remembers the operation that produced it in `.grad_fn`, and `grad_fn.next_functions` points one step back towards the leaves.

```
x = torch.tensor(2.0, requires_grad=True)
w = torch.tensor(0.5, requires_grad=True)
f = torch.sin(w * x) + x ** 2
print(f, f.grad_fn)
print(f.grad_fn.next_functions)
```

```
tensor(4.8415, grad_fn=<AddBackward0>)
 <AddBackward0 object at 0x000001EDB67A7EE0>
((<SinBackward0 object at 0x000001EDB07F0B20>, 0), (<PowBackward0 object at 0x000001EDB8
2B0AF0>, 0))
```

The chain rule, backwards

Each node knows its own local derivative (the derivative of $\sin$ is $\cos$, of a product with respect to one factor is the other factor, ...). The chain rule combines them. For a scalar output it is cheapest to go **backwards**, from the output to the leaves: one backward pass gives the derivative of f with respect to *every* leaf at about the cost of one forward pass. This is **reverse-mode** differentiation, called **backpropagation** for neural networks. For the example:

$$\frac{\partial f}{\partial w} = \cos(wx)\, x, \qquad \frac{\partial f}{\partial x} = \cos(wx)\, w + 2x.$$

Note that x reaches f along two paths; the chain rule adds their contributions.

```
f.backward()                        # fills .grad of every leaf with requires_grad=True
print("autograd:", w.grad.item(), x.grad.item())
print("by hand: ", (torch.cos(w * x) * x).item(), (torch.cos(w * x) * w + 2 * x).item())
```

```
autograd: 1.0806046724319458 4.270151138305664
by hand:  1.0806046724319458 4.270151138305664
```

The rules you need

- `loss.backward()` computes the gradients of `loss` (which must be a scalar) with respect to all leaves with `requires_grad=True` and stores them in their `.grad` attribute.
- `torch.autograd.grad(loss, [a, b])` returns the gradients as a tuple instead of storing them.
- **Gradients accumulate.** `backward()` *adds* to `.grad`; it does not overwrite. This is deliberate (it lets you sum gradients over several losses), but it means you must reset the gradients before each training step: `w.grad = None`, or `optimizer.zero_grad()` (Section 1.3).
- **The graph is freed after a backward pass.** Calling `backward()` a second time on the same result raises an error. Either recompute the result (this builds a new graph), or pass `retain_graph=True` to the first call.
- `with torch.no_grad():` switches graph building off. Use it for evaluation and for updating parameters by hand; an update of a parameter should not itself become part of the graph.
- `t.detach()` returns the same values without the graph; `t.item()` turns a one-element tensor into a Python number.

```
w.grad = None
for k in range(3):
    loss = (w * 3.0) ** 2          # a new graph each time; d loss / dw = 18 w = 9
    loss.backward()
    print("call", k + 1, "w.grad =", w.grad.item())
w.grad = None                      # reset

with torch.no_grad():
    z = w * 3.0
print("built inside no_grad:", z.requires_grad, z.grad_fn)
```

```
call 1 w.grad = 9.0
call 2 w.grad = 18.0
call 3 w.grad = 27.0
built inside no_grad: False None
```

Logistic regression and its loss

Task 1.2 uses logistic regression with one feature: a linear score passed through the **sigmoid** function, which maps $\mathbb{R}$ to $(0, 1)$, so the output can be read as the probability of class 1:

$$z = w_1 x_1 + b, \qquad a = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

The loss is the **binary cross-entropy**, the negative log-likelihood of the label $y \in \{0, 1\}$ under a Bernoulli distribution with parameter a:

$$L = -\big[\, y \log a + (1 - y) \log(1 - a) \,\big].$$

A useful identity for the derivation: $\sigma'(z) = \sigma(z)\,(1 - \sigma(z))$.

Checking gradients numerically

An analytic gradient, yours or a library's, should always be compared with a numerical approximation. Taylor expansion of f around θ gives

$$f(\theta + h) = f(\theta) + h f'(\theta) + \tfrac{h^2}{2} f''(\theta) + \tfrac{h^3}{6} f'''(\theta) + \dots$$

The **forward difference** $\big(f(\theta+h)-f(\theta)\big)/h$ therefore has error $\frac{h}{2}f''(\theta)+\cdots=O(h)$. The **central difference**

$$f'(\theta)\approx\frac{f(\theta+h)-f(\theta-h)}{2h}$$

cancels the even terms and has error $\frac{h^2}{6}f'''(\theta)+\cdots=O(h^2)$. This is the **truncation error**: the error of the formula in exact arithmetic. It suggests making h as small as possible.

But the computer does not use exact arithmetic. Each value $f(\theta\pm h)$ is computed with a relative rounding error of order ε (Section 1.1), so its absolute error is about $\varepsilon\,|f(\theta)|$. The difference of two nearly equal numbers keeps this absolute error while the difference itself becomes tiny (**cancellation**), and the division by $2h$ then magnifies it. This is the **rounding error** of the finite difference. Task 1.2 asks you to measure how the two errors combine.

To compare two gradients use the **relative error** $\|g-\tilde g\|/\max(\|g\|,\|\tilde g\|)$: an absolute error of 10^{-6} means something different for a gradient of size 10^{-3} than for one of size 10^{3}.

PyTorch you need. `requires_grad=True`, `.backward()`, `torch.autograd.grad`, `.grad`, `retain_graph=True`, `torch.no_grad()`, `.detach()`, `.item()`; `dtype=torch.float64`. A log–log plot: `plt.loglog(hs, errors, "o-", label="float64")`, then `plt.legend()` and `plt.show()`. A logarithmic grid: `hs = 10.0 ** -np.arange(1, 13)`.

1.3 Linear regression by gradient descent

The model and the loss

Linear regression predicts $\hat y=\mathbf{w}^\top\mathbf{x}+b$. With N examples stacked as rows of $X\in\mathbb{R}^{N\times d}$, the **mean squared error** is

$$\hat R(\mathbf{w},b)=\frac{1}{N}\sum_{i=1}^{N}\big(\mathbf{w}^\top\mathbf{x}_i+b-y_i\big)^2.$$

The bias is easiest to handle by appending a column of ones: with $\tilde X=[X\ \ \mathbf{1}]\in\mathbb{R}^{N\times(d+1)}$ and $\theta=(\mathbf{w},b)\in\mathbb{R}^{d+1}$,

$$\hat R(\theta)=\frac{1}{N}\|\tilde X\theta-\mathbf{y}\|^2.$$

The exact solution

Setting the gradient to zero gives the **normal equations** $\tilde X^\top\tilde X\,\theta=\tilde X^\top\mathbf{y}$. Solving them directly squares the condition number of $\tilde X$, so numerical libraries solve the least-squares problem with a QR or SVD factorisation instead: `np.linalg.lstsq(A, y, rcond=None)[0]` or `torch.linalg.lstsq(A, y).solution`.

Gradients of vector expressions

For the gradient-descent version you need $\nabla_\theta \hat{R}$ in matrix form. Two rules for vectors $\theta, \mathbf{a}$ and a matrix A are enough:

$$\nabla_\theta \left(\mathbf{a}^\top \theta\right) = \mathbf{a}, \qquad \nabla_\theta \left(\theta^\top A\, \theta\right) = \left(A + A^\top\right) \theta.$$

Expand $\|\tilde{X}\theta - \mathbf{y}\|^2 = (\tilde{X}\theta - \mathbf{y})^\top (\tilde{X}\theta - \mathbf{y})$ and apply them. The result has a natural reading: the residuals $\tilde{X}\theta - \mathbf{y}$, weighted by the inputs.

Gradient descent

Gradient descent starts somewhere, here at $\theta_0 = \mathbf{0}$, and repeats

$$\theta_{k+1} = \theta_k - \eta\, \nabla_\theta \hat{R}(\theta_k),$$

with **learning rate** (step size) $\eta > 0$. **Full-batch** gradient descent uses all N examples in every step; the mini-batch version of Section 2.2 uses a few at a time.

The same algorithm in NumPy and in PyTorch, on a toy problem, minimising $f(\mathbf{u}) = \|\mathbf{u} - \mathbf{c}\|^2$ with gradient $2(\mathbf{u} - \mathbf{c})$:

```
c = np.array([3.0, -1.0])

# NumPy: you write the gradient
u = np.zeros(2)
for _ in range(50):
    grad = 2 * (u - c)
    u -= 0.1 * grad
print("NumPy:           ", u)

# PyTorch, update by hand: autograd computes the gradient
ct = torch.tensor(c)
u = torch.zeros(2, dtype=torch.float64, requires_grad=True)
for _ in range(50):
    loss = ((u - ct) ** 2).sum()
    loss.backward()
    with torch.no_grad():
        u -= 0.1 * u.grad
    u.grad = None
print("PyTorch by hand:", u.detach().numpy())

# PyTorch with an optimiser: it does the update and the reset
u = torch.zeros(2, dtype=torch.float64, requires_grad=True)
optimizer = torch.optim.SGD([u], lr=0.1)
for _ in range(50):
    loss = ((u - ct) ** 2).sum()
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
print("torch.optim.SGD:", u.detach().numpy())
```

```
NumPy:           [ 2.99995718 -0.99998573]
PyTorch by hand: [ 2.99995718 -0.99998573]
torch.optim.SGD: [ 2.99995718 -0.99998573]
```

An **optimiser** (`torch.optim.SGD` , `torch.optim.Adam` , ...) receives the list of tensors it should update. `optimizer.zero_grad()` resets their gradients and `optimizer.step()` applies the

update rule, for SGD exactly $\theta \leftarrow \theta - \eta \nabla_\theta L$. The tensors can be any leaves with `requires_grad=True`, not only the parameters of a model.

How large can the learning rate be?

For a quadratic loss the analysis is exact. The Hessian of $\hat{R}$ is constant, $H = \frac{2}{N}\tilde{X}^\top \tilde{X}$, symmetric and positive semi-definite. Writing $e_k = \theta_k - \theta^*$ for the distance to the minimiser, one step of gradient descent gives

$$e_{k+1} = (I - \eta H)\, e_k.$$

In the eigenbasis of H this decouples into one scalar recursion per eigenvalue λ_i. Work out for which η every component shrinks: that is the theoretical bound Task 1.3 asks for. The same recursion also shows why convergence is slow when the eigenvalues differ a lot. The **condition number** $\lambda_{\max}/\lambda_{\min}$ grows when the features have very different scales, which is why features are usually standardised.

Random numbers

Make experiments reproducible with a fixed seed: in NumPy `rng = np.random.default_rng(0)` and then `rng.standard_normal((N, d))`; in PyTorch `torch.manual_seed(0)`, or a local `g = torch.Generator().manual_seed(0)` passed as `generator=g`. Samples with different standard deviations per feature: multiply standard normal samples by a vector of standard deviations, which broadcasts over the rows.

PyTorch you need. `np.linalg.lstsq`, `np.linalg.eigvalsh` (eigenvalues of a symmetric matrix; `torch.linalg.eigvalsh` in PyTorch), `np.hstack` / `torch.cat` to append the column of ones; `torch.optim.SGD([theta], lr=...)`, `zero_grad`, `step`; `torch.tensor(a)` keeps NumPy's `float64`; `plt.semilogy` for loss curves.

1.4 A multilayer network

Layers

A **linear layer** (fully connected layer) maps $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ to $W\mathbf{x} + \mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$. In PyTorch, `nn.Linear(d_in, d_out)` stores `weight` of shape $(d_{\text{out}}, d_{\text{in}})$ and `bias` of shape $(d_{\text{out}},)$, so it has $d_{\text{in}} d_{\text{out}} + d_{\text{out}}$ parameters. It works on a whole **batch** at once: the input has shape (B, d_{in}), one example per row, and the layer computes $XW^\top + \mathbf{b}$ of shape (B, d_{out}). In PyTorch the batch is always the first dimension.

A composition of linear maps is again linear, so stacking linear layers alone gains nothing. A **multilayer perceptron** (MLP) puts a non-linear **activation** between the layers, most often $\mathrm{ReLU}(u) = \max(0, u)$, applied entry by entry. With $\mathbf{h}^{(0)} = \mathbf{x}$:

$$\mathbf{h}^{(\ell)} = \mathrm{ReLU}\left(W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}\right), \qquad \ell = 1, \dots, L-1, \qquad \mathbf{z} = W^{(L)}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}.$$

The last layer has no activation. Its outputs $\mathbf{z} \in \mathbb{R}^C$ are the **logits**, one unnormalised score per class.

Modules

A model is a subclass of `nn.Module`. You create the layers in `__init__` (after calling `super().__init__()`) and describe the computation in `forward`. Calling `model(x)` runs `forward`; autograd provides the backward pass. Layers assigned as attributes are registered automatically, so `model.parameters()` finds all their parameters. `nn.Sequential(layer1, layer2, ...)` chains layers that run one after another; `nn.Sequential(*a_list)` does the same for a Python list.

```
class TwoLayer(nn.Module):
    def __init__(self, d_in, d_hidden, d_out):
        super().__init__()
        self.hidden = nn.Linear(d_in, d_hidden)
        self.out = nn.Linear(d_hidden, d_out)

    def forward(self, x):
        return self.out(torch.relu(self.hidden(x)))


torch.manual_seed(0)
net = TwoLayer(4, 8, 2)
print(net)
for name, p in net.named_parameters():
    print(f"{name:15s} {tuple(p.shape)}  {p.numel()} entries, trainable: {p.requires_grad
print(net(torch.randn(5, 4)).shape)     # a batch of 5 examples
```

```
TwoLayer(
  (hidden): Linear(in_features=4, out_features=8, bias=True)
  (out): Linear(in_features=8, out_features=2, bias=True)
)
hidden.weight   (8, 4)  32 entries, trainable: True
hidden.bias     (8,)  8 entries, trainable: True
out.weight      (2, 8)  16 entries, trainable: True
out.bias        (2,)  2 entries, trainable: True
torch.Size([5, 2])
```

The weights start as small random numbers (PyTorch uses a uniform distribution scaled by $1/\sqrt{d_{\text{in}}}$). If all weights started equal, every unit of a layer would compute the same function, receive the same gradient and stay identical forever: the random start **breaks the symmetry**.
`torch.manual_seed(s)` before creating the model makes the initialisation reproducible. Setting `p.requires_grad_(False)` freezes a parameter; you will use this for fine-tuning in List 6.

From logits to probabilities

The **softmax** turns logits into a probability distribution:

$$\operatorname{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{C} e^{z_j}}.$$

It is positive, sums to 1, and keeps the order of the logits, so the most likely class is $\arg\max_k z_k$ either way. Computed literally it is fragile. The largest `float32` number is about $3.4 \times 10^{38} \approx e^{88.7}$, so e^z overflows to `inf` for $z > 88.7$, and for very negative logits all exponentials underflow to 0. The fix rests on one identity: for any constant c,

$$\operatorname{softmax}(\mathbf{z}) = \operatorname{softmax}(\mathbf{z} - c\,\mathbf{1}),$$

because the factor e^{-c} cancels between numerator and denominator. Task 1.4 asks you to choose c well and explain the choice. The same trick inside the logarithm gives the stable **log-sum-exp**, which loss functions use internally; this is why models return logits and leave the softmax to the loss.

PyTorch you need. `nn.Module`, `super().__init__()`, `forward`, `nn.Linear`, `nn.ReLU()` (or `torch.relu`), `nn.Sequential(*layers)`, `model.parameters()`, `model.named_parameters()`, `p.numel()`, `p.requires_grad`; `z.max(dim=-1, keepdim=True).values`, `torch.exp`, `torch.allclose`.

Lab 2: Data Loading, the Training Loop & a Bigram Language Model

2.1 `Dataset` and `DataLoader`

Why batches

Computing the loss and its gradient over the whole data set in every step is expensive for large data, and unnecessary: the average over a random **mini-batch** of, say, 32 examples is an unbiased estimate of the full gradient. Training therefore loops over the data in batches; one full pass over the data is an **epoch**.

The two classes

PyTorch splits data handling in two:

- A **`Dataset`** knows how to produce **one** example. A custom one subclasses `torch.utils.data.Dataset` and implements `__len__(self)` (the number of examples) and `__getitem__(self, i)` (example `i`, usually a pair `(features, label)` of tensors). Where the data come from is up to you: a tensor in memory, files on disk, generated on the fly.
- A **`DataLoader`** draws indices, calls `__getitem__` for each, and **stacks** the results into batches: examples of shape $(d,)$ become a batch of shape (B, d), scalar labels a batch of shape $(B,)$.

```
class Squares(Dataset):
    # example i: the number i as a feature, i**2 as the target
    def __init__(self, n):
        self.x = torch.arange(n, dtype=torch.float32)

    def __len__(self):
        return len(self.x)

    def __getitem__(self, i):
        return self.x[i : i + 1], self.x[i] ** 2      # features of shape (1,), a scalar
```

```
ds = Squares(10)
print(len(ds), ds[3])
torch.manual_seed(0)
for epoch in range(2):
    print("epoch", epoch, [xb[:, 0].tolist() for xb, yb in DataLoader(ds, batch_size=4, s
```

```
10 (tensor([3.]), tensor(9.))
epoch 0 [[6.0, 7.0, 1.0, 4.0], [2.0, 0.0, 9.0, 8.0], [3.0, 5.0]]
epoch 1 [[2.0, 4.0, 7.0, 0.0], [8.0, 9.0, 5.0, 3.0], [6.0, 1.0]]
```

The most important `DataLoader` arguments:

argument	meaning
`batch_size`	examples per batch; the last batch is smaller if N is not a multiple of it
`shuffle=True`	a new random order in every epoch (use it for training data)
`drop_last=True`	skip the incomplete last batch
`num_workers`	background processes for loading; keep 0 in notebooks, especially on Windows

Class labels must be integers $0, \dots, C-1$ of type `int64` (`torch.long`).

Splitting the data

- The **training set** is used to fit the parameters.
- The **validation set** is used to make choices: learning rate, number of epochs, which model to keep.
- The **test set** is used **once**, at the end, to estimate how the final model does on new data. Using it for any choice makes the estimate optimistic.

`torch.utils.data.random_split(dataset, [0.6, 0.2, 0.2], generator=torch.Generator().manual_seed(0))` returns three subsets that behave like datasets. `TensorDataset(X, y)` is a ready-made `Dataset` for data already in tensors.

The default data: Gaussian blobs

Four classes in the plane, each a cloud of points around its centre $\boldsymbol{\mu}_c$: $\mathbf{x} = \boldsymbol{\mu}_c + \sigma\boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I)$, generated with `torch.randn(n, 2)`. Labels: `torch.arange(4).repeat_interleave(n)` gives $0, \dots, 0, 1, \dots, 1, \dots$. The classes overlap a little, so a perfect classifier does not exist, which makes the comparison of models meaningful.

PyTorch you need. `Dataset`, `__len__`, `__getitem__`, `DataLoader(ds, batch_size=..., shuffle=...)`, `random_split`, `TensorDataset`; `torch.cat`, `repeat_interleave`; `plt.scatter(x, y, c=labels)`.

2.2 The training loop

The loss for classification

For C classes the model returns logits $\mathbf{z} \in \mathbb{R}^C$. The **cross-entropy** loss is the negative log-probability that the softmax assigns to the correct class y:

$$\ell(\mathbf{z}, y) = -\log \operatorname{softmax}(\mathbf{z})_y = -z_y + \log \sum_{j=1}^{C} e^{z_j}.$$

It is 0 when the model puts probability 1 on the correct class and grows without bound as that probability goes to 0. A model that knows nothing and outputs equal logits has loss $\ln C$, a useful sanity check for the first batch. Binary cross-entropy (Section 1.2) is the special case $C = 2$.

`F.cross_entropy(logits, targets)` takes logits of shape (N, C), **not** probabilities, and integer targets of shape $(N,)$, and returns the **mean** over the N examples. With `reduction="sum"` it returns the sum. For sequence models (List 5) the logits have shape (B, T, C) and the targets (B, T): flatten them first with `logits.reshape(-1, C)` and `targets.reshape(-1)`, so every position counts as one prediction.

The loop

Every training loop in this course has the same structure:

```
for epoch in range(num_epochs):
    model.train()
    for features, labels in train_loader:
        logits = model(features)                  # 1. forward pass: builds the
graph
        loss = F.cross_entropy(logits, labels)    # 2. mean loss over the batch
        optimizer.zero_grad()                     # 3. reset the accumulated
gradients
        loss.backward()                           # 4. gradients of the loss
w.r.t. every parameter
        optimizer.step()                          # 5. update the parameters
    model.eval()
    # evaluate on validation data, inside torch.no_grad()
```

Forgetting step 3 is the classic bug: gradients from all previous batches keep adding up (Section 1.2). `model.train()` and `model.eval()` switch layers that behave differently during training and evaluation, such as dropout in List 3; an MLP has none, but call them anyway so the code stays correct later.

Evaluating over a whole loader

The loss and the accuracy on a data set are averages over **all examples**. Batches can differ in size (the last one is often smaller), so averaging per-batch means is wrong in general. Instead, accumulate the sum of the losses (`reduction="sum"`) and the number of examples over all batches, and divide once at the end. The same holds for the accuracy: count correct predictions, $\hat{y} = \arg\max_k z_k$ (`logits.argmax(dim=-1)`), and examples. Evaluation runs inside `torch.no_grad()` : no graph is needed, which saves memory and time.

Optimisers

SGD takes the step $\theta \leftarrow \theta - \eta\, \mathbf{g}$ with the mini-batch gradient $\mathbf{g}$. **Adam** keeps running averages of the gradient and of its square for every parameter,

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\,\mathbf{g}_t, \qquad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\,\mathbf{g}_t^2, \qquad \theta_t = \theta_{t-1} - \eta\,\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon},$$

where $\hat{\mathbf{m}}_t = \mathbf{m}_t/(1-\beta_1^t)$ and $\hat{\mathbf{v}}_t = \mathbf{v}_t/(1-\beta_2^t)$ correct the bias of the zero initialisation (defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$). Each parameter gets its own effective step size, so Adam is much less sensitive to the scale of the gradients. Typical learning rates: 0.01–0.1 for SGD on small problems, 10^{-4}–10^{-2} for Adam. Swapping the optimiser changes one line; the loop stays the same.

Overfitting and model selection

The training loss keeps falling as the model adapts to the training data, including its noise. The validation loss falls at first and then may rise again: the model starts to **overfit**. Plot both curves per epoch. Decisions (learning rate, optimiser, number of epochs) are made on the validation set; the test set is evaluated once for the final choice.

Saving and loading

A model's learned state is its `state_dict`: an ordered dictionary from parameter names to tensors. It contains no code, so to restore a model you first construct the same architecture, then load the tensors into it:

```
torch.save(net.state_dict(), "example.pth")
print({k: tuple(v.shape) for k, v in net.state_dict().items()})

copy = TwoLayer(4, 8, 2)                                    # same architecture, new random
copy.load_state_dict(torch.load("example.pth", weights_only=True))
x = torch.randn(3, 4)
print(torch.equal(net(x), copy(x)))

import os
os.remove("example.pth")
```

```
{'hidden.weight': (8, 4), 'hidden.bias': (8,), 'out.weight': (2, 8), 'out.bias': (2,)}
True
```

Devices

The reusable functions take a `device` argument so that the same code runs on a GPU later: move the model once with `model.to(device)` (before creating the optimiser), and every batch inside the loop with `features.to(device)`, `labels.to(device)`. With `device="cpu"` these calls cost nothing.

PyTorch you need. `F.cross_entropy` (with `reduction="sum"`), `logits.argmax(dim=-1)`, `model.train()`, `model.eval()`, `torch.no_grad()`, `torch.optim.SGD`, `torch.optim.Adam`, `optimizer.zero_grad()`, `loss.backward()`, `optimizer.step()`, `loss.item()`; `torch.save`, `torch.load(..., weights_only=True)`, `load_state_dict`, `torch.equal`.

2.3 A count-based bigram language model

Language models

A **language model** assigns a probability to every sequence of **tokens** $x_1, \ldots, x_T$. Tokens are the units the text is cut into: here single characters; in List 2, sub-word pieces as in GPT. By the chain rule of probability, *any* distribution over sequences can be written as a product of next-token distributions:

$$p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_1, \ldots, x_{t-1}).$$

So a language model is a machine for predicting the next token from the previous ones. A GPT does exactly this with the whole preceding context. The simplest possible version looks at one previous token only.

Tokens as integers

Models work with numbers, so each distinct character gets an integer ID. The set of distinct tokens is the **vocabulary**, of size V.

```
sample = "the cat sat on the mat"
vocab = sorted(set(sample))
stoi = {ch: i for i, ch in enumerate(vocab)}          # string to integer
itos = {i: ch for ch, i in stoi.items()}              # integer to string
ids = torch.tensor([stoi[ch] for ch in sample])
print(vocab, len(vocab))
print(ids)
print("".join(itos[i] for i in ids.tolist()))
```

```
[' ', 'a', 'c', 'e', 'h', 'm', 'n', 'o', 's', 't'] 10
tensor([9, 4, 3, 0, 2, 1, 9, 0, 8, 1, 9, 0, 7, 6, 0, 9, 4, 3, 0, 5, 1, 9])
the cat sat on the mat
```

The bigram model

A **bigram** model makes the **Markov assumption** that the next token depends only on the previous one, $p(x_t \mid x_{<t}) = p(x_t \mid x_{t-1})$. Then the whole model is a $V \times V$ table P with $P_{ij} = p(j \mid i)$; every row is a probability distribution.

Let $c(i, j)$ be the number of times token j follows token i in the training text. Maximising the likelihood of the training text over all tables with rows summing to 1 (a Lagrange-multiplier exercise) gives the **relative frequencies**

$$P_{ij} = \frac{c(i, j)}{\sum_k c(i, k)}.$$

This **maximum-likelihood estimate** assigns probability 0 to every pair that never occurs in the training text. A text is never long enough to contain every plausible pair, so **smoothing** moves a little probability to the unseen pairs. The simplest form, **add-α smoothing**, pretends every pair was seen α extra times:

$$P_{ij} = \frac{c(i, j) + \alpha}{\sum_k c(i, k) + \alpha V}.$$

$\alpha = 1$ is add-one (Laplace) smoothing; as $\alpha \to \infty$ every row becomes uniform.

Perplexity

A language model is evaluated by how probable it finds held-out text. The average **negative log-likelihood** per predicted token is

$$\mathrm{NLL} = -\frac{1}{T-1}\sum_{t=2}^{T}\log p(x_t \mid x_{t-1}),$$

which is exactly the mean cross-entropy of Section 2.2. Its exponential is the **perplexity**,

$$\mathrm{PPL} = e^{\mathrm{NLL}} = \Big(\prod_{t=2}^{T} p(x_t \mid x_{t-1})\Big)^{-1/(T-1)},$$

the inverse geometric mean of the probabilities the model gave to the tokens that actually came. A model that always spreads its probability evenly over k tokens, one of them the right one, has perplexity k; so perplexity reads as "the model is as uncertain as if it chose uniformly among PPL tokens". Lower is better; 1 is perfect.

For text, take the validation set as the **end** of the text rather than random characters: neighbouring characters are strongly dependent, and a random split would leak information from training into validation.

Counting and evaluating without loops

Every pair (i, j) can be turned into one integer $iV + j \in \{0, \dots, V^2 - 1\}$. `torch.bincount` counts how often each integer occurs, and `.view(V, V)` turns the counts back into a table. **Advanced indexing** with two index tensors picks out one table entry per pair: `P[a, b]` with `a`, `b` of length n returns the n values `P[a[0], b[0]], P[a[1], b[1]], …`.

```
V = len(vocab)
counts = torch.bincount(ids[:-1] * V + ids[1:], minlength=V * V).view(V, V)
print("pairs starting with 't':", {itos[j]: counts[stoi['t'], j].item() for j in range(V)

P = (counts + 1.0) / (counts + 1.0).sum(dim=1, keepdim=True)        # add-one smoothing
probs = P[ids[:-1], ids[1:]]                                         # probability of eve
print("perplexity on its own text:", math.exp(-torch.log(probs).mean().item()))
```

```
pairs starting with 't': {' ': 2, 'h': 2}
perplexity on its own text: 5.010243554500344
```

Generating text

To generate, start from some token, sample the next one from its row of P, and repeat. `torch.multinomial(p, num_samples=1)` draws an index with probabilities `p`; pass `generator=torch.Generator().manual_seed(0)` for reproducible samples.

```
g = torch.Generator().manual_seed(0)
cur, out = stoi["t"], ["t"]
for _ in range(30):
    cur = torch.multinomial(P[cur], 1, generator=g).item()
```

```
    out.append(itos[cur])
print("".join(out))
```

```
tnateosahee msah s oeama mata h
```

The default text

The Verdict by Edith Wharton (1908, public domain), about 20,000 characters: the text the book uses, which you will meet again in Lists 2 and 5. Read it with `Path("the-verdict.txt").read_text(encoding="utf-8")`. Download: https://raw.githubusercontent.com/rasbt/LLMs-from-scratch/main/ch02/01_main-chapter-code/the-verdict.txt

PyTorch you need. `sorted(set(text))`, dictionaries for `stoi` / `itos`; `torch.bincount(..., minlength=...)`, `.view(V, V)`, `.sum(dim=1, keepdim=True)`, advanced indexing `P[a, b]`, `torch.log`, `math.exp`; `torch.multinomial`.

2.4 A neural bigram language model

The same model, learned by gradient descent

Replace the table of probabilities by a table of **logits** $W \in \mathbb{R}^{V \times V}$ and define

$$p_W(j \mid i) = \mathrm{softmax}(W_i)_j,$$

where W_i is row i. Every row can represent any distribution with positive entries, so this is the same family of models as in Section 2.3, parametrised differently. Instead of counting, the parameters are learned by minimising the cross-entropy on the training pairs (x_{t-1}, x_t) with the training loop of Section 2.2. Because the mean cross-entropy is the NLL of Section 2.3, the validation perplexity is simply $e^{\text{validation loss}}$.

Embeddings

`nn.Embedding(num_embeddings, embedding_dim)` is a trainable lookup table: a weight matrix of shape `(num_embeddings, embedding_dim)`, and calling it with a tensor of integer IDs returns the corresponding rows. Mathematically, looking up row i is the same as multiplying the one-hot vector $\mathbf{e}_i$ by the matrix, only much cheaper. With `nn.Embedding(V, V)` each token ID is mapped to its row of V logits, which is exactly W_i. In GPT the same layer maps token IDs to vectors of a smaller size d.

```
emb = nn.Embedding(5, 3)
print(emb.weight.shape)
idx = torch.tensor([4, 0, 4])
print(emb(idx))                                  # rows 4, 0, 4 of the weight mat
one_hot = F.one_hot(idx, num_classes=5).float()
print(torch.allclose(emb(idx), one_hot @ emb.weight))    # lookup = one-hot times matrix
```

```
torch.Size([5, 3])
tensor([[-0.0052, -0.0944, -1.1236],
        [-1.9513,  0.4186,  3.3214],
        [-0.0052, -0.0944, -1.1236]], grad_fn=<EmbeddingBackward0>)
True
```

`nn.Embedding` initialises its weights from $\mathcal{N}(0, 1)$; `nn.init.zeros_(emb.weight)` sets them all to zero instead.

Training data

Each training example is one pair: input the ID of a character, target the ID of the next one. `TensorDataset(ids[:-1], ids[1:])` produces exactly these pairs, and your `train` function from Task 2.2 works unchanged: the model maps a batch of B IDs to logits of shape (B, V), and the targets have shape $(B,)$.

Regularisation

The count model needs α because maximum likelihood fits the training text too closely. A neural model has its own ways of not fitting the training data perfectly:

- **Early stopping**: stop training when the validation loss stops improving.
- **Weight decay**: penalise large weights. Adding $\frac{\lambda}{2}||\theta||^2$ to the loss adds $\lambda\theta$ to the gradient, which pulls every weight towards 0 in every step. For Adam the decoupled version, `torch.optim.AdamW(params, lr=..., weight_decay=λ)`, is the standard; it shrinks the weights by the factor $1 - \eta\lambda$ per step, so λ has to be fairly large (around 10^{-2}) to matter when the number of steps is small.
- **The initialisation** itself: where training starts determines where a finite number of steps ends.

Why this matters for the course

Next-token prediction with the cross-entropy loss is the pretraining objective of GPT. Over the semester only the function that produces the logits changes: here a table lookup that sees one character, in List 4 a transformer that sees the whole context.

PyTorch you need. `nn.Embedding`, `nn.init.zeros_`, `TensorDataset`, `math.log`, `math.exp`; `torch.optim.AdamW(..., weight_decay=...)`; your `train` and `compute_loss`.

2.5 Ideas for your own experiment

About one hour of work; one plot or table and a paragraph is enough.

- Go from bigrams to trigrams, by counting or with a small network that sees two previous characters. How much does the perplexity drop, and what happens to the gap between training and validation?
- Use words instead of characters as tokens. What happens to V, to the count table, and to the role of smoothing?
- Compare the perplexity of the same model on two languages or two authors. What does the difference measure?
- Sample from the bigram model with a temperature, $p \propto p^{1/\tau}$. How does the text change?
- Implement Adam or momentum yourself and check it against `torch.optim`.
- Batch size against learning rate: at equal numbers of epochs, which combinations train well?

- Initialise all weights of the MLP to the same constant. What does it learn, and why?
- Time a large matrix product on the CPU and on a GPU (Colab) for growing sizes. Where is the break-even?

Appendix A: The four reusable functions

`count_parameters` (List 1, Task 1.4) and `compute_loss`, `compute_accuracy`, `train` (Task 2.2) are called again in Lists 4–7, so their signatures are fixed. Full specification:

```
def count_parameters(model: nn.Module, trainable_only: bool = True) -> int:
    """Number of parameters of a model.

    Args:
        model: any PyTorch module.
        trainable_only: if True, count only parameters with requires_grad=True.

    Returns:
        The total number of scalar parameters.
    """
    ...


def compute_loss(model: nn.Module, loader: DataLoader, device: torch.device |
str = "cpu") -> float:
    """Mean cross-entropy of a model over a whole data loader.

    The loader yields (inputs, targets). The model maps inputs to logits whose
last
    dimension is the number of classes. Logits may have shape (B, C) with
targets (B,),
    or (B, T, C) with targets (B, T), as in the GPT of List 5.

    The mean is taken over all predictions in the loader, not over batches, so
the
    result does not depend on the batch size. Runs in eval mode without
gradients.

    Returns:
        The mean cross-entropy as a Python float. exp of it is the perplexity.
    """
    ...


def compute_accuracy(model: nn.Module, loader: DataLoader, device: torch.device
| str = "cpu") -> float:
    """Fraction of correct argmax predictions over a whole data loader.

    Same conventions as compute_loss: logits (B, C) with targets (B,).
    Correct predictions and examples are summed over all batches and divided
once.

    Returns:
```

```
        The accuracy in [0, 1] as a Python float.
    """
    ...


def train(
    model: nn.Module,
    train_loader: DataLoader,
    val_loader: DataLoader,
    optimizer: torch.optim.Optimizer,
    num_epochs: int,
    device: torch.device | str = "cpu",
) -> dict[str, list[float]]:
    """Train a model with cross-entropy and evaluate it after every epoch.

    For each epoch: put the model in train mode, loop over train_loader
    (forward, loss, zero_grad, backward, step), then call compute_loss on
val_loader.
    Batches are moved to `device`; the model is expected to be on `device`
already.

    Returns:
        A dictionary with two lists of length num_epochs:
        "train_loss": the mean of the batch losses of each epoch,
        "val_loss":   compute_loss(model, val_loader, device) after each epoch.
    """
    ...
```

To check your implementations, run this after defining them:

```
import math
from torch.utils.data import TensorDataset

_g = torch.Generator().manual_seed(0)
_x = torch.randn(40, 3, generator=_g)
_y = (_x[:, 0] > 0).long()
_net = nn.Sequential(nn.Linear(3, 8), nn.ReLU(), nn.Linear(8, 2))
_loader = DataLoader(TensorDataset(_x, _y), batch_size=16)

assert count_parameters(_net) == 3 * 8 + 8 + 8 * 2 + 2
_net[0].weight.requires_grad_(False)
assert count_parameters(_net) == 26 and count_parameters(_net,
trainable_only=False) == 50
_net[0].weight.requires_grad_(True)

_l16 = compute_loss(_net, _loader)
_l40 = compute_loss(_net, DataLoader(TensorDataset(_x, _y), batch_size=40))
assert isinstance(_l16, float) and abs(_l16 - _l40) < 1e-5, "the loss must not
depend on the batch size"
assert 0.0 <= compute_accuracy(_net, _loader) <= 1.0

# Logits of shape (B, T, C) with targets (B, T), as in List 5
_seq = nn.Embedding(5, 5)
nn.init.zeros_(_seq.weight)
_tok = torch.randint(0, 5, (12, 7), generator=_g)
assert abs(compute_loss(_seq, DataLoader(TensorDataset(_tok, _tok),
batch_size=4)) - math.log(5)) < 1e-5
```

```
_h = train(_net, _loader, _loader, torch.optim.Adam(_net.parameters(), lr=0.05),
num_epochs=20)
assert set(_h) == {"train_loss", "val_loss"} and len(_h["train_loss"]) ==
len(_h["val_loss"]) == 20
assert _h["val_loss"][-1] < _h["val_loss"][0] and compute_accuracy(_net,
_loader) > 0.9
print("Contract check passed: count_parameters, compute_loss, compute_accuracy,
train")
```